



University of Asia Pacific

Department of Computer Science and Engineering

CSE 450: VLSI Lab

LAB REPORT

Experiment Number: 02

Experiment Title: Design and Simulation of a 1-Bit Full Adder Using VHDL in Xilinx ISE 14.7

Submitted by:

Name: Taposhi Rabeya Bashar

Student ID: 22201060

Section: C

Submitted to:

Md. Rasheduzzaman

Lecturer

Department of Computer Science and Engineering

Date of Submission: 20-09-2026

1. Title of the Experiment

Design and Simulation of a 1-Bit Full Adder Using VHDL in Xilinx ISE 14.7

2. Objectives

The main objective of this experiment is to design a 1-bit full adder using the VHDL hardware description language and to verify its functionality through simulation in the Xilinx ISE Design Suite 14.7. A full adder is a basic combinational circuit that adds three single-bit inputs, namely two operands (A and B) and a carry-in (Cin), and produces a sum (Sum) and a carry-out (Cout). Through this experiment, we aim to understand the structure of a VHDL design (entity and architecture), to write a testbench that applies every possible input combination, and to use the ISim simulator to check the output waveform against the theoretical truth table. The experiment also introduces the workflow of creating a project, adding source files, running behavioral simulation, and viewing the synthesized RTL schematic.

3. Theoretical Background

Introduction to Adders

In digital electronics, arithmetic operations such as addition are performed using combinational logic circuits, whose outputs depend only on the present inputs and not on any previous state. The simplest of these circuits are the half adder and the full adder. A half adder adds two single-bit numbers and produces a sum and a carry, but it cannot accept a carry from a previous stage. Because of this limitation, a half adder alone cannot be used to build multi-bit adders.

Full Adder

A full adder overcomes this limitation by adding three single-bit inputs: the two operand bits (A and B) and a carry-in (Cin) that comes from the previous, less significant stage. It produces two outputs: a sum bit (Sum) and a carry-out (Cout) that is passed to the next, more significant stage. Since the addition of three bits can give a result between 0 and 3 (00 to 11 in binary), two output bits are required to represent it. The Sum output is the least significant bit of the result, and Cout is the most significant bit.

The behaviour of the full adder follows a simple rule. The Sum output is 1 when an odd number of inputs are 1, and the Cout output is 1 when at least two of the inputs are 1.

Table 1: Truth table of the full adder

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0

A	B	Cin	Sum	Cout
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Boolean Expressions

From the truth table, the output equations are obtained as follows:

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

$$\text{Cout} = AB + \text{BCin} + \text{ACin}$$

The carry-out can also be written in an equivalent form that reflects the two-half-adder construction:

$$\text{Cout} = AB + \text{Cin}(A \oplus B)$$

Both forms give identical results for all input combinations.

Implementation Using Two Half Adders

A full adder can be built from two half adders and one OR gate. The first half adder adds A and B, producing an intermediate sum $S1 = A \oplus B$ and an intermediate carry $C1 = A \cdot B$. The second half adder adds S1 and Cin, producing the final Sum = $S1 \oplus \text{Cin}$ and a second carry $C2 = S1 \cdot \text{Cin}$. The two carries are then combined by an OR gate to give $\text{Cout} = C1 + C2$. In total, this implementation uses two XOR gates, two AND gates and one OR gate.

Ripple-Carry Adder

Multiple full adders can be cascaded to add multi-bit binary numbers. In an n-bit ripple-carry adder, the carry-out of each full adder is connected to the carry-in of the next one. Although this structure is simple, the carry must propagate through every stage, so the total delay increases with the number of bits. Faster designs, such as the carry look-ahead adder, are used when speed is important.

Hardware Description in VHDL

In this experiment, the full adder is described in VHDL using an entity, which defines the input and output ports (a, b, cin, sum and cout), and an architecture, which describes the logic using the Boolean expressions given above. The design is verified by a testbench that applies all eight possible input combinations and by comparing the simulated output with the truth table.

4. Hardware and Software Requirements

1. Xilinx ISE Design Suite 14.7 (Project Navigator)
2. ISim Simulator (Lite version)
3. XST (Xilinx Synthesis Technology)
4. Oracle VirtualBox
5. Virtual Machine: ISE_14.7_VM_base (Oracle Linux, 64-bit)

6. Windows 11 (host operating system)
7. VHDL (hardware description language)
8. GitHub (for hosting the code repository)

5. RTL-Schematic Diagrams

The RTL schematic shows how the synthesis tool interprets the VHDL description as a network of logic gates and interconnections. The design has three inputs (a, b, cin) and two outputs (sum, cout). The sum output is implemented as the XOR of all three inputs, which is realized by two cascaded XOR gates: the first XORs a and b, and its result is XORed with cin. The carry-out is implemented with the majority function, $(a \text{ AND } b) \text{ OR } (b \text{ AND } \text{cin}) \text{ OR } (a \text{ AND } \text{cin})$, which appears in the schematic as AND gates whose outputs are combined by OR logic. The top-level view shows the design as a single block named `full_adder`, and the expanded internal view shows the individual gates and the nets connecting them.

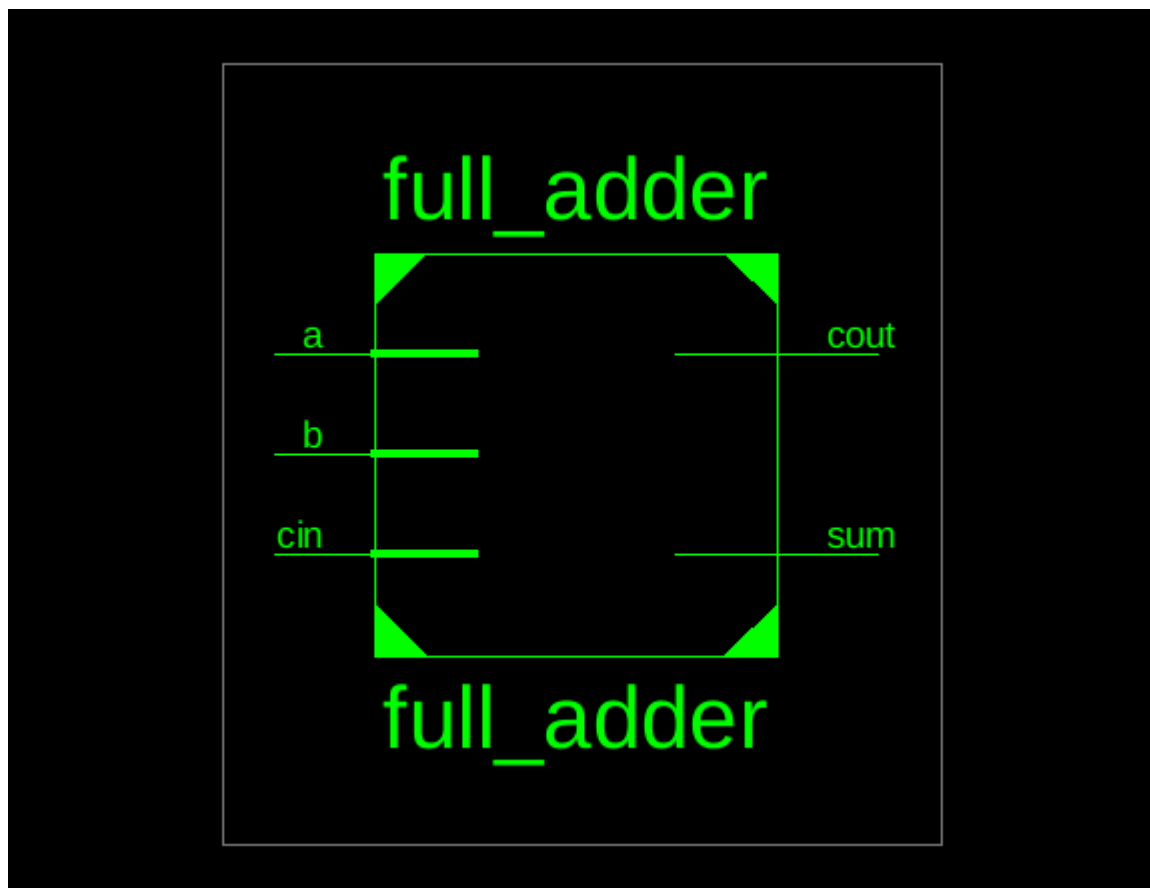


Figure 1: Top-level RTL schematic of the 1-bit full adder

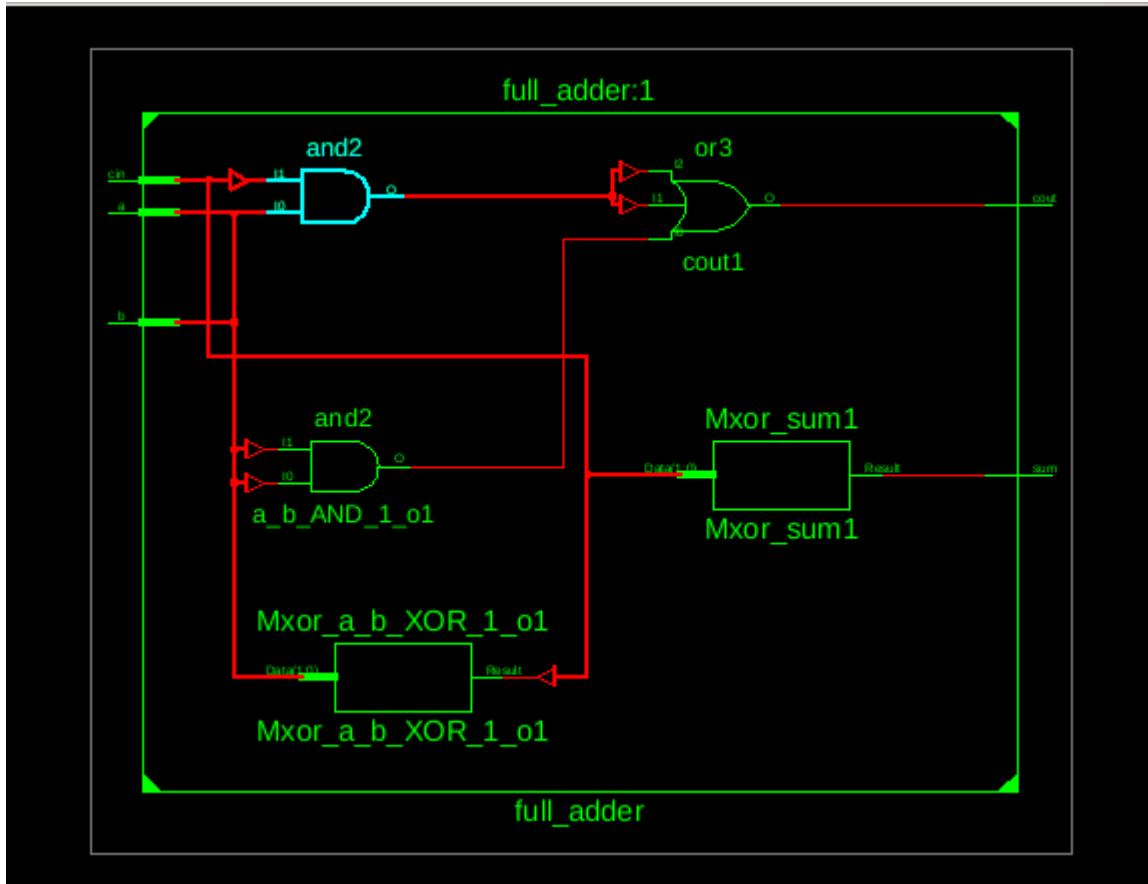


Figure 2: Internal RTL schematic of the 1-bit full adder

6. Simulations

The design was verified using behavioral simulation in the ISim simulator. A VHDL testbench instantiated the full adder as the unit under test and applied all eight possible combinations of the inputs a, b and cin, each held for 10 ns, starting after an initial delay of 100 ns. The resulting waveform shows the inputs changing every 10 ns between 100 ns and 180 ns, with the sum and cout outputs responding immediately to each change, as expected for a combinational circuit. The sum output is high only when an odd number of inputs are high, and the cout output is high whenever at least two inputs are high. Comparing the simulated waveform with the theoretical truth table in Table 2, all eight cases matched, which confirms that the design is functionally correct. After 180 ns, the stimulus ends and the signals hold their last values ($a = b = cin = 1$, $sum = 1$, $cout = 1$).

Table 2: Simulated truth table of the 1-bit full adder

Time (ns)	a	b	cin	sum	cout
100–110	0	0	0	0	0
110–120	0	0	1	1	0
120–130	0	1	0	1	0
130–140	0	1	1	0	1

Time (ns)	a	b	cin	sum	cout
140–150	1	0	0	1	0
150–160	1	0	1	0	1
160–170	1	1	0	0	1
170–180	1	1	1	1	1

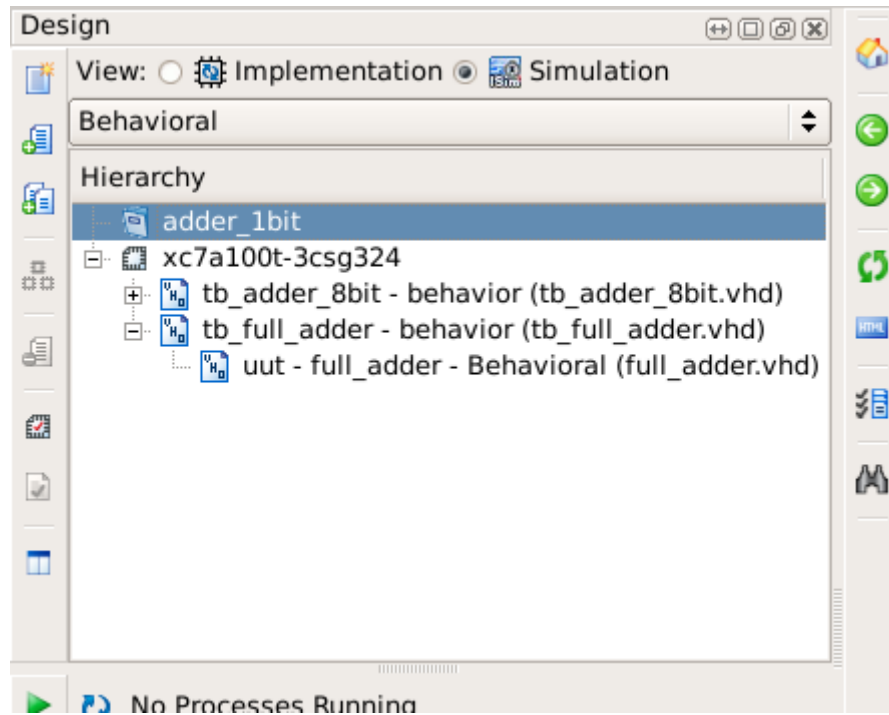


Figure 3: Project hierarchy in ISE Project Navigator

```

71     b => b,
72     cin => cin,
73     sum => sum,
74     cout => cout
75 );
76
77
78
79 -- Stimulus process
80 stim_proc: process
81 begin
82     -- hold reset state for 100 ns.
83     wait for 100 ns;
84
85     a <= '0'; b <= '0'; cin <= '0'; wait for 10 ns;
86     a <= '0'; b <= '0'; cin <= '1'; wait for 10 ns;
87     a <= '0'; b <= '1'; cin <= '0'; wait for 10 ns;
88     a <= '0'; b <= '1'; cin <= '1'; wait for 10 ns;
89     a <= '1'; b <= '0'; cin <= '0'; wait for 10 ns;
90     a <= '1'; b <= '0'; cin <= '1'; wait for 10 ns;
91     a <= '1'; b <= '1'; cin <= '0'; wait for 10 ns;
92     a <= '1'; b <= '1'; cin <= '1'; wait for 10 ns;
93
94     wait;
95 end process;
96
97 END;
98

```

Figure 4: Testbench stimulus process

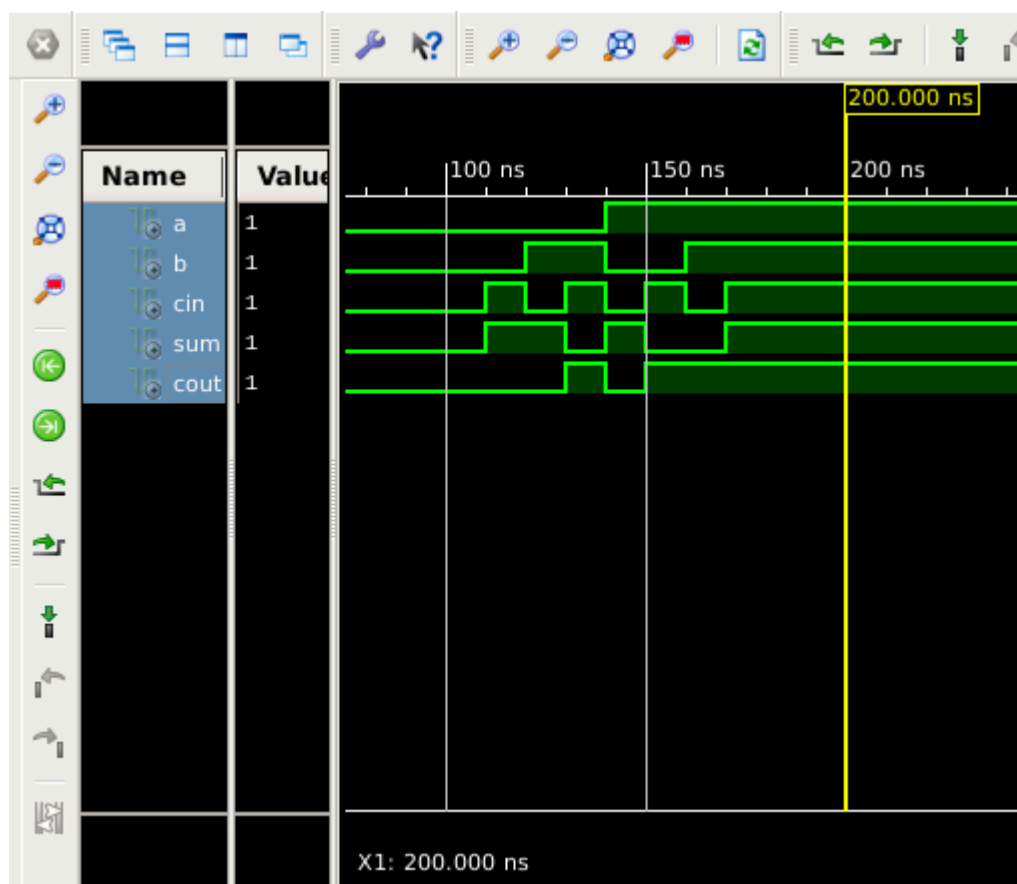


Figure 5: Simulation waveform of the 1-bit full adder

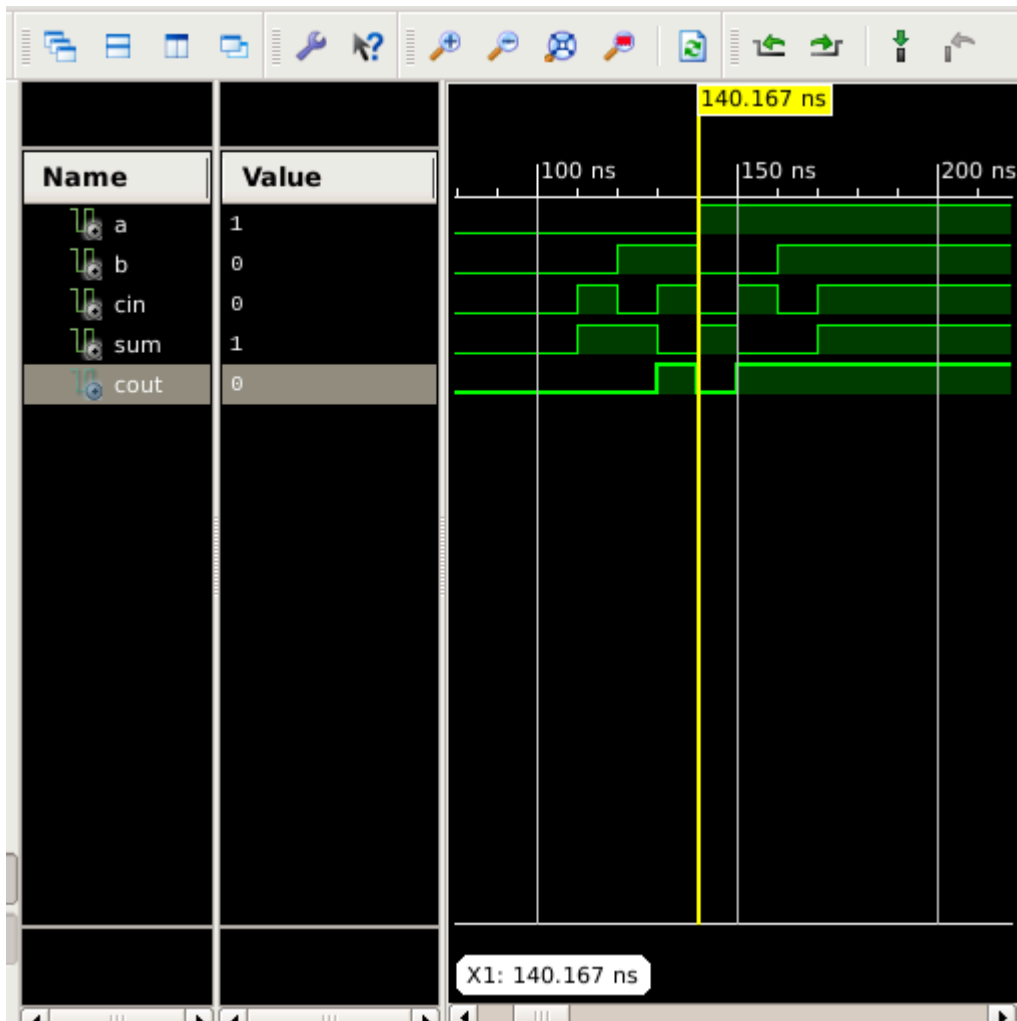


Figure 6: Waveform with cursor placed

Figure 7: GitHub repository containing the source files

Design code (full_adder.vhd):

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity full_adder is
    Port ( a      : in  STD_LOGIC;
          b      : in  STD_LOGIC;
          cin     : in  STD_LOGIC;
          sum     : out STD_LOGIC;
          cout    : out STD_LOGIC);
end full_adder;

architecture Behavioral of full_adder is
begin
    sum <= a xor b xor cin;
    cout <= (a and b) or (b and cin) or (a and cin);
end Behavioral;
```

Testbench stimulus (tb_full_adder.vhd, inside stim_proc):

```
wait for 100 ns;

a <= '0'; b <= '0'; cin <= '0'; wait for 10 ns;
a <= '0'; b <= '0'; cin <= '1'; wait for 10 ns;
a <= '0'; b <= '1'; cin <= '0'; wait for 10 ns;
a <= '0'; b <= '1'; cin <= '1'; wait for 10 ns;
a <= '1'; b <= '0'; cin <= '0'; wait for 10 ns;
a <= '1'; b <= '0'; cin <= '1'; wait for 10 ns;
a <= '1'; b <= '1'; cin <= '0'; wait for 10 ns;
a <= '1'; b <= '1'; cin <= '1'; wait for 10 ns;

wait;
```

7. Result

The 1-bit full adder was successfully designed in VHDL and simulated in the Xilinx ISE 14.7 environment using the ISim simulator. The testbench applied all eight possible combinations of the inputs a, b and cin, each held for 10 ns after an initial delay of 100 ns. In every case, the simulated Sum and Cout outputs matched the values in the theoretical truth table. The Sum output was high only when an odd number of inputs were high, and the Cout output was high whenever at least two inputs were high. Synthesis with XST completed without errors, and the generated RTL schematic agreed with the intended logic structure of the design.

The objectives of the experiment were achieved. The full adder was described using a VHDL entity and architecture, and a testbench was written to apply every input combination. The output waveform was verified against the truth table, and the RTL schematic was viewed after synthesis.

8. Code in GitHub Repository

GitHub repository link: <https://github.com/<your-username>/<repository-name>>

9. Conclusion

In this experiment, a 1-bit full adder was successfully designed in VHDL and simulated using the Xilinx ISE 14.7 design suite. The design was described at the behavioral level using Boolean expressions, with the sum computed as the XOR of the three inputs and the carry-out computed by the majority function. A testbench was written to apply all eight input combinations, and the ISim waveform was compared with the theoretical truth table. All outputs matched the expected values, confirming that the design works correctly. The RTL schematic generated after synthesis also agreed with the intended logic structure of two XOR gates for the sum and AND-OR logic for the carry. During the experiment, we also learned to resolve practical issues such as removing unused clock placeholders from the auto-generated testbench and setting the correct source language for the project. This experiment provides a foundation for building larger arithmetic circuits, such as ripple-carry adders, from 1-bit full adder blocks.